Sur ce PDF, vous trouverez comptes rendus :

TP3 Compte rendu Champ d'Application **1**
Partie I: Génération de composants et styles SCSS 1
Partie II: Ajout d'interactions 6
TP5 Compte rendu Champ d'Application **16**

TP3 Compte rendu Champ d'Application

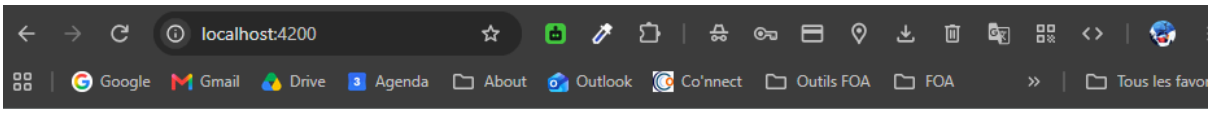
Romain SIEURIN - 18/09/2026

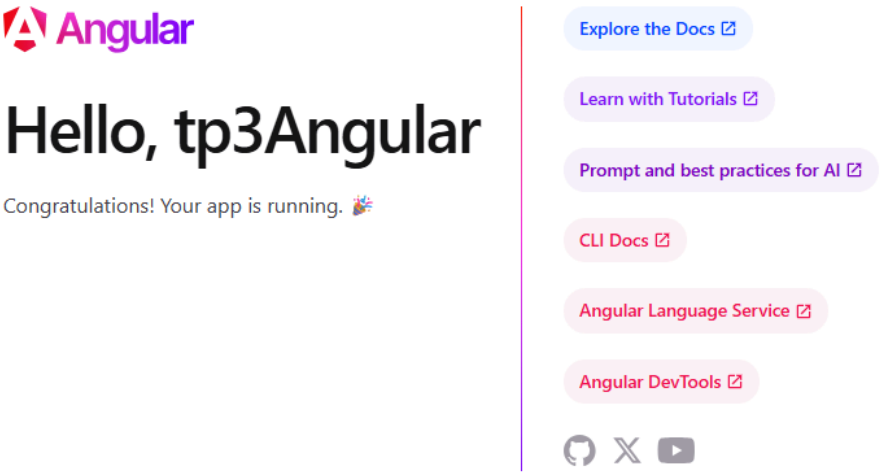
Partie I: Génération de composants et styles SCSS

1.1 Créez un nouveau projet que vous nommerez **ModuleAngularTP3**. Pendant l'installation, choisissez le format de style **SCSS**.

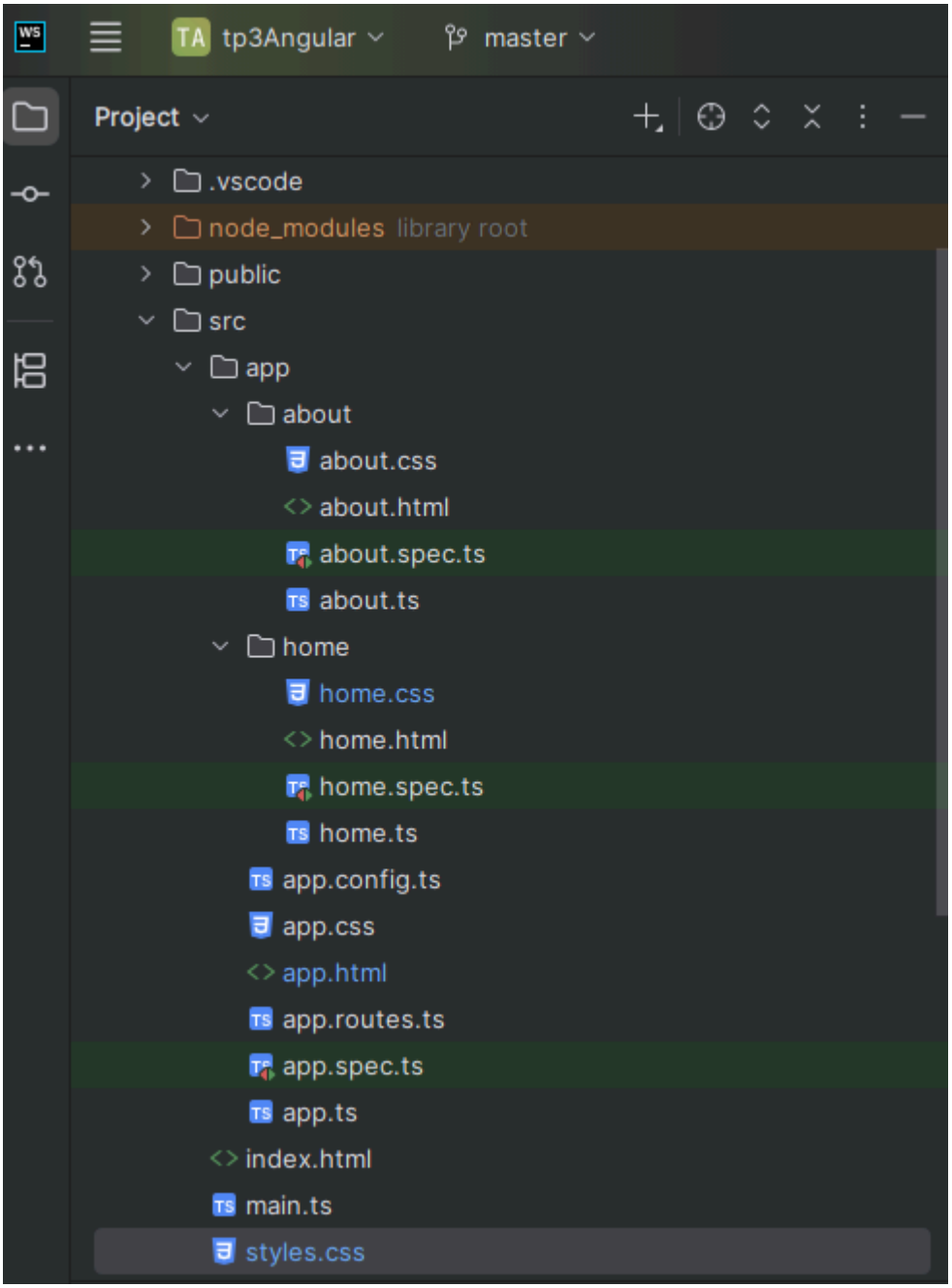
```
(base) PS C:\Master\champApplication\tp3> ng new tp3Angular
✔Which stylesheet system would you like to use? CSS             [ https://developer.mozilla.org/docs/Web/CSS
        ]
✔Do you want to enable Server-Side Rendering (SSR) and Static Site Generation (SSG/Prerendering)? No
✔Which AI tools do you want to configure with Angular best practices? https://angular.dev/ai/develop-with-ai None
CREATE tp3Angular/.prettierrc (173 bytes)
CREATE tp3Angular/angular.json (1988 bytes)
CREATE tp3Angular/package.json (788 bytes)
CREATE tp3Angular/README.md (1523 bytes)
CREATE tp3Angular/tsconfig.json (990 bytes)
CREATE tp3Angular/.editorconfig (331 bytes)
CREATE tp3Angular/.gitignore (666 bytes)
CREATE tp3Angular/tsconfig.app.json (444 bytes)
CREATE tp3Angular/tsconfig.spec.json (456 bytes)
CREATE tp3Angular/.vscode/extensions.json (134 bytes)
CREATE tp3Angular/.vscode/launch.json (490 bytes)
CREATE tp3Angular/.vscode/mcp.json (188 bytes)
CREATE tp3Angular/.vscode/tasks.json (1020 bytes)
CREATE tp3Angular/src/main.ts (228 bytes)
CREATE tp3Angular/src/index.html (309 bytes)
CREATE tp3Angular/src/styles.css (81 bytes)
CREATE tp3Angular/src/app/app.spec.ts (700 bytes)
CREATE tp3Angular/src/app/app.ts (304 bytes)
CREATE tp3Angular/src/app/app.css (0 bytes)
CREATE tp3Angular/src/app/app.html (20488 bytes)
CREATE tp3Angular/src/app/app.config.ts (324 bytes)
CREATE tp3Angular/src/app/app.routes.ts (80 bytes)
CREATE tp3Angular/public/favicon.ico (15086 bytes)
npm error Cannot read properties of null (reading 'edgesOut')
npm error A complete log of this run can be found in: C:\Users\Sieur\AppData\Local\npm-cache\_logs\2026-09-18T07_19_27_1
32Z-debug-0.log
```

1.2 Lancez le serveur et vérifiez que vous obtenez bien la même page de démarrage que pour les séances précédentes.





1.3 Créez deux nouveaux composants *home* et *about*.



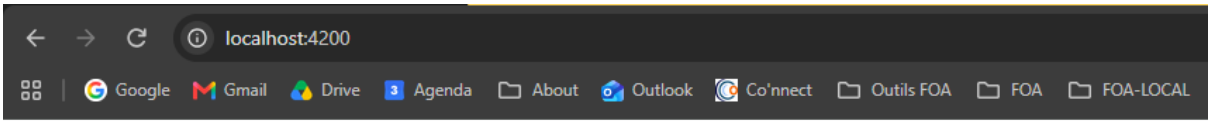
1.4 Pour utiliser ces composants dans les composants de l'application racine, `App`, il est nécessaire de rajouter les deux composants dans les imports.

ME.md | index.html | app.html | app.css | styles.css | home.css | home.ts

```ts
import { Component, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { About } from './about/about';
import { Home } from './home/home';

@Component({ Show usages  srn-romain
  selector: 'app-root',
  imports: [RouterOutlet, Home, About],
  templateUrl: './app.html',
  styleUrl: './app.css'
})
export class App {
  protected readonly title :WritableSignal<string> = signal( initialValue: 'tp3Angular');
}
```

1.5 Supprimez tout le contenu de la page par défaut du fichier `app.html` sauf la ligne `<router-outlet />`. Rajoutez une `unordered-list` avec un item défini comme un lien de type : `<a href="">Accueil</a>`. Faites un second item avec un second lien vers *About* (qui contiendra le lien "about"). Enfin, rajoutez une balise pour faire afficher par défaut votre composant app-home.
A noter : pour l'instant les liens ne fonctionneront pas mais vous devriez avoir une visualisation ressemblant à l'image ci-dessous.



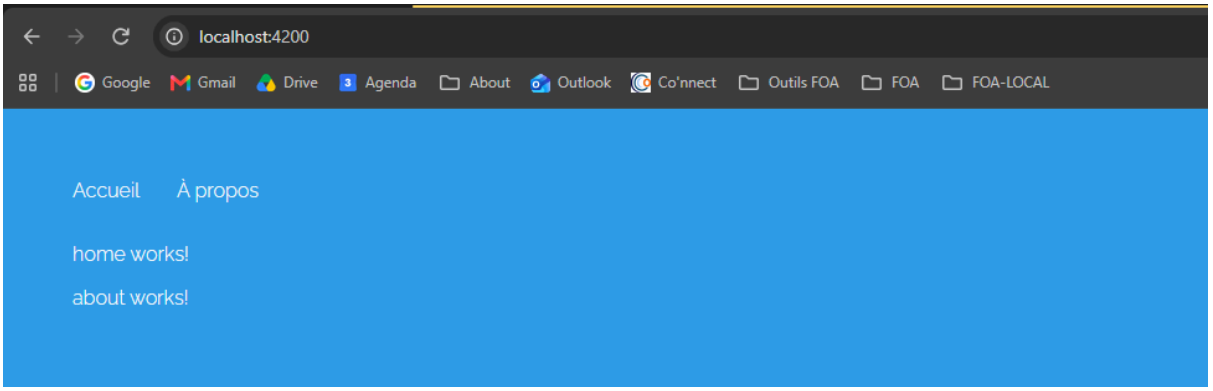
- Accueil
- À propos

home works!

about works!

1.6 Recopiez les styles css suivants dans leur fichiers respectifs. Ces styles sont accessibles à l'adresse suivante sur *moodle*. Vous devriez obtenir une page ressemblant à l'image suivante (a).

J'ai bien comme l'image demandé :

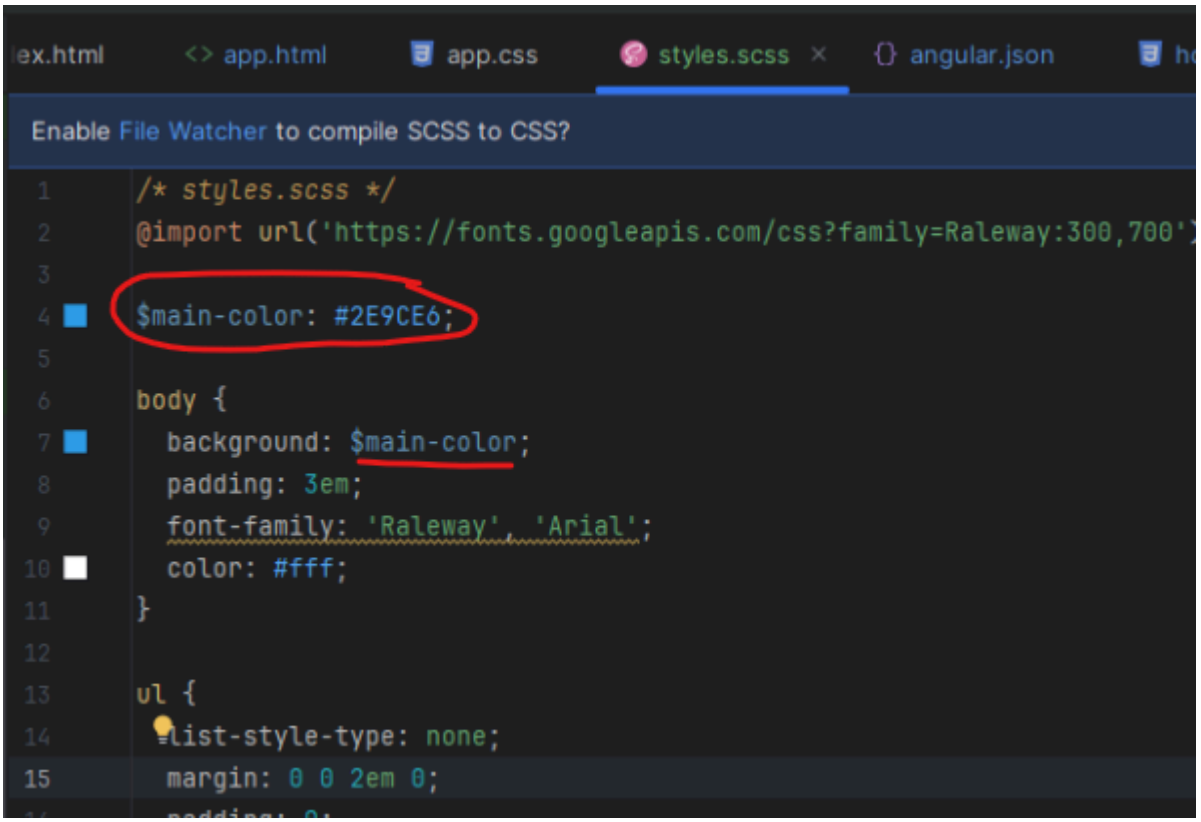


1.7 Utilisation de variables SCSS. Le fichier `styles.scss` contient plusieurs couleurs utilisées par l'application. SCSS permet de définir des variables afin d'éviter de répéter directement les valeurs des couleurs dans les règles CSS.

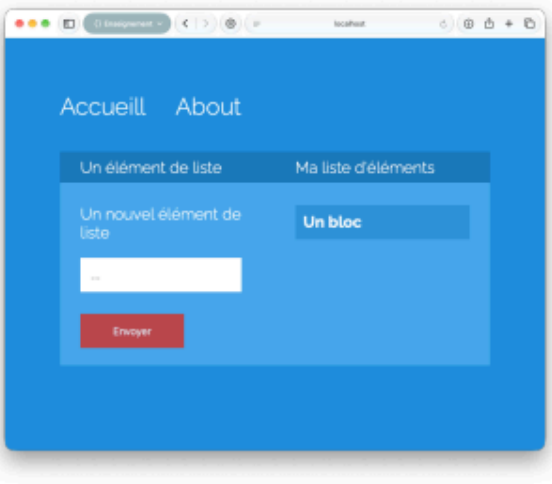
Dans le fichier `styles.scss`, définissez une variable `$main-color` contenant la couleur bleue utilisée pour le fond de la page :

```
$main-color: #2E9CE6;
```

Utilisez ensuite cette variable à la place de la valeur `#2E9CE6` dans la règle `body`.
Modifiez ensuite la variable pour observer directement le changement de couleur de la page.



1.8 Modifiez le code du fichier `home.html` de façon à obtenir la visualisation suivante :

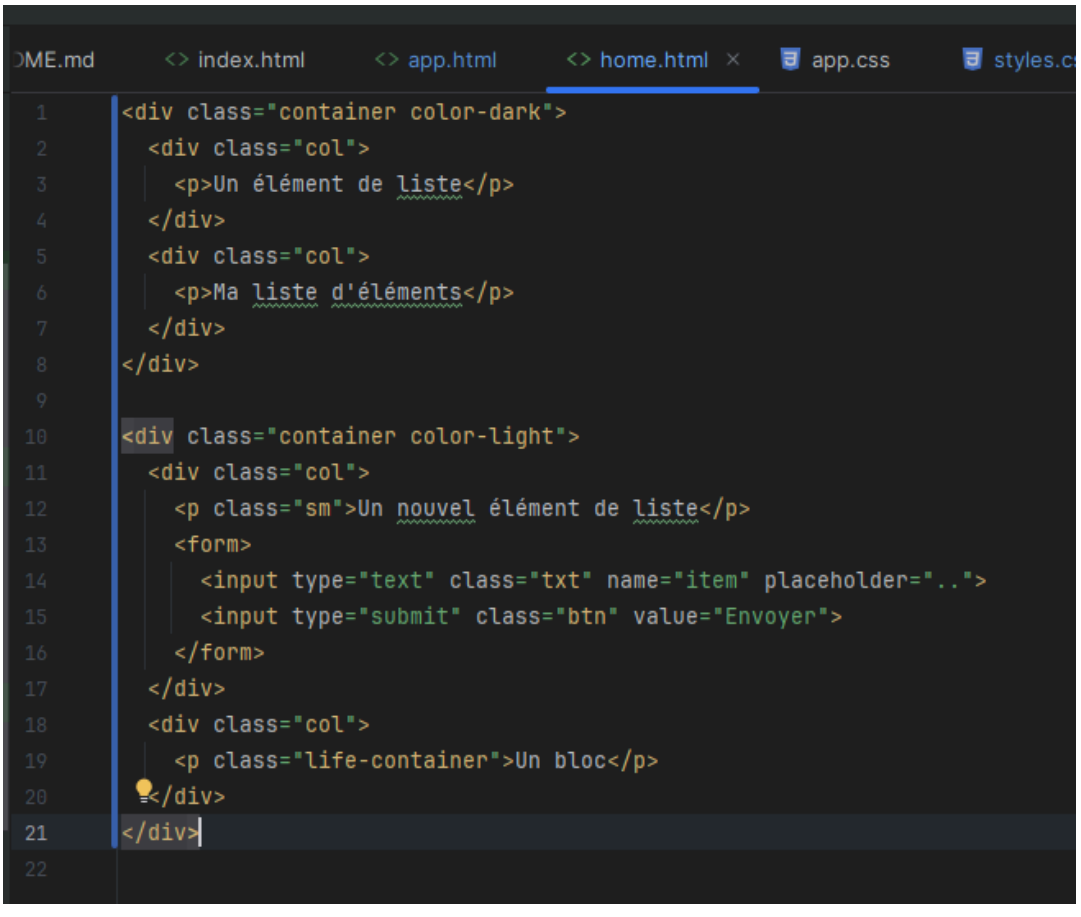


Indications : le source doit contenir les éléments suivants :

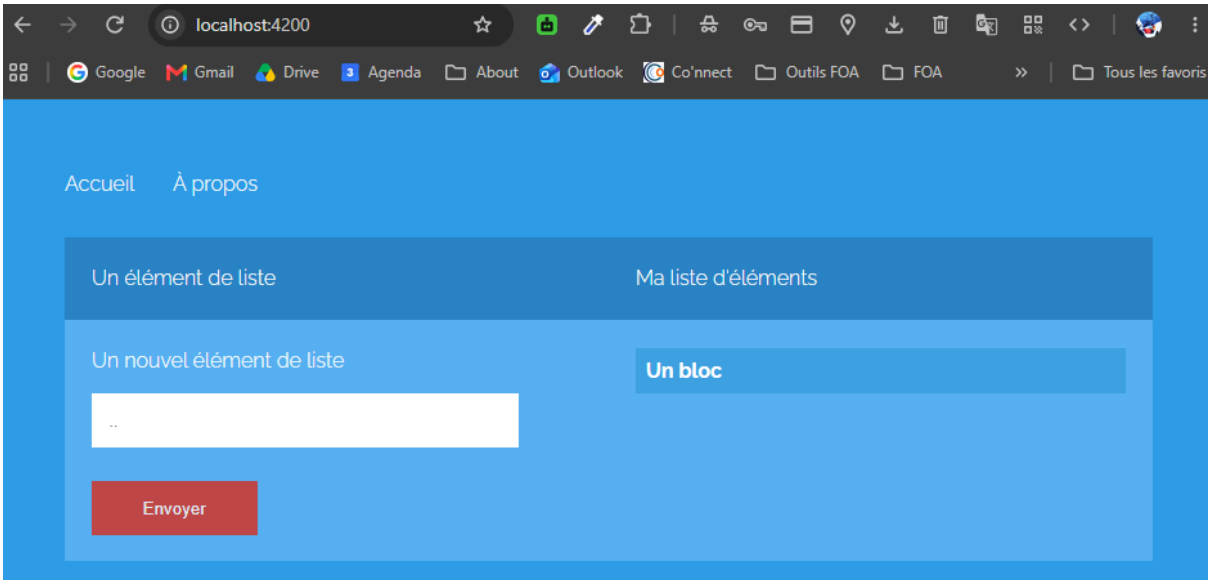
- `<div class="container color-dark">`
- `<div class="container color-light">`
- `<div class="col">`
- `<p class="life-container">`
- Un formulaire contenant :
  - une balise input de type *text* de classe `txt`
  - une balise input de type submit de classe `btn`



J'ai donc recopié le code de la consigne dans mon fichier home.html :



Et cela me donne :

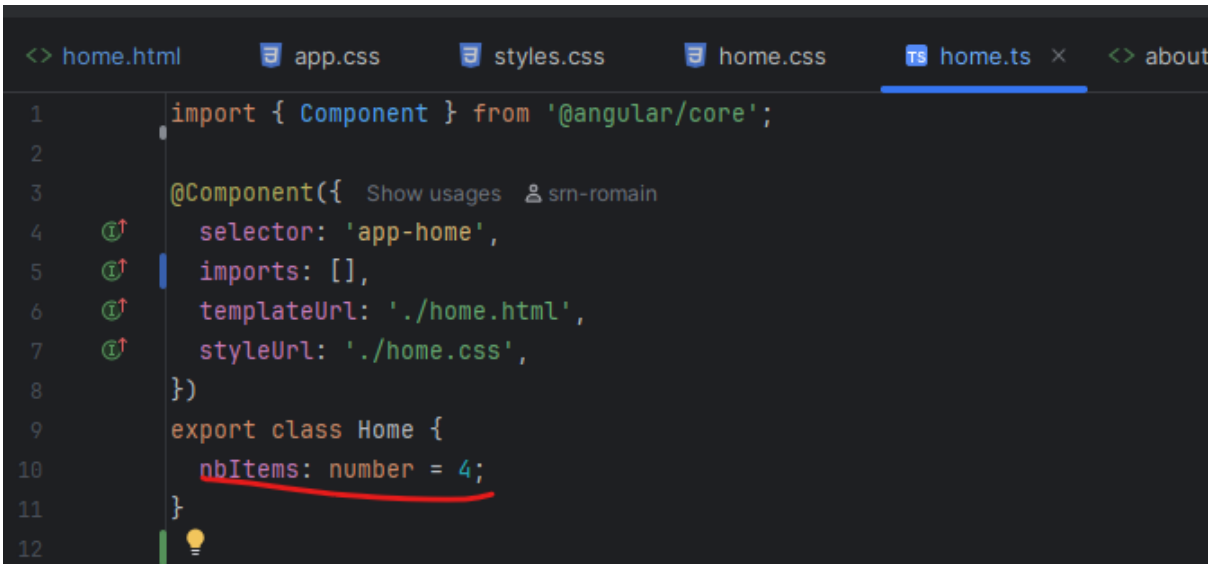


Partie II: Ajout d'interactions

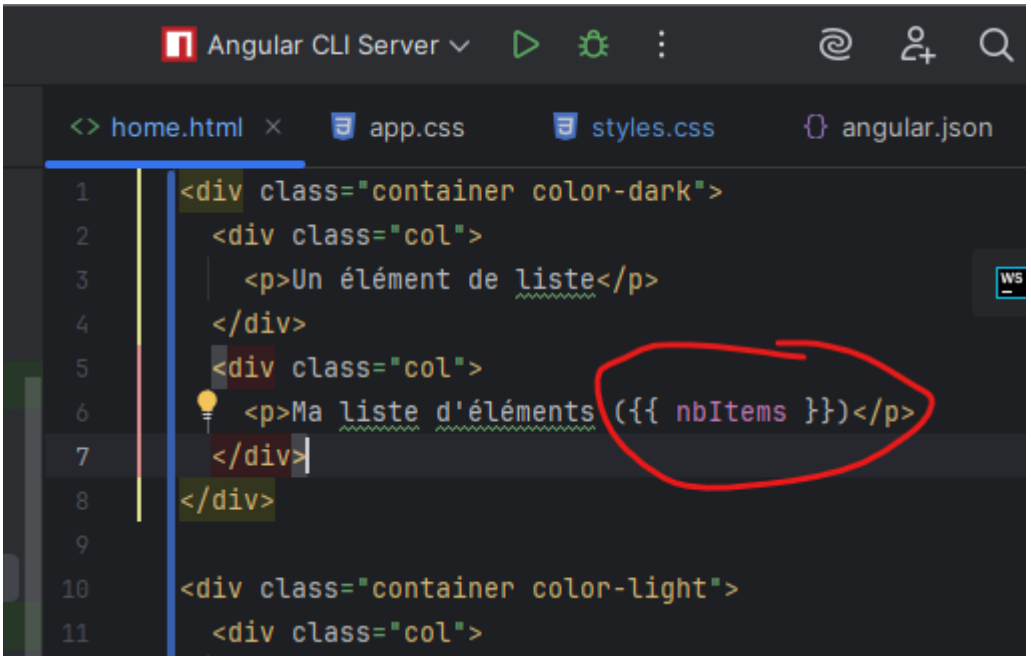
2.1 Un premier élément dynamique de l'application est d'afficher le nombre de taches que l'utilisateur aura saisie dans l'interface. Pour cela dans le fichier `home.ts`, rajoutez une variable `nbItems` de type `number` initialisée à 4 et qui devra s'afficher à la suite du titre *Ma liste d'éléments* comme sur l'image suivante :



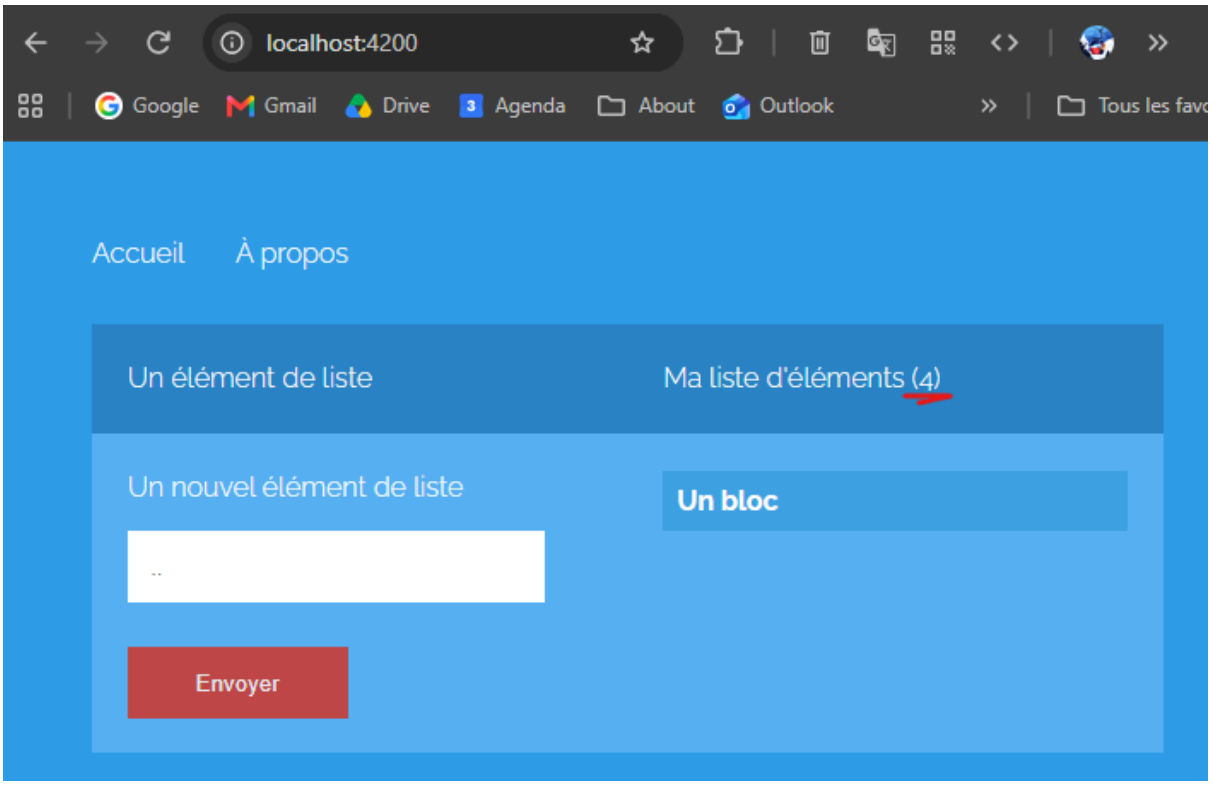
J'initialise la variable dans le home.ts :



Et dans home.html je rajoute la variable :

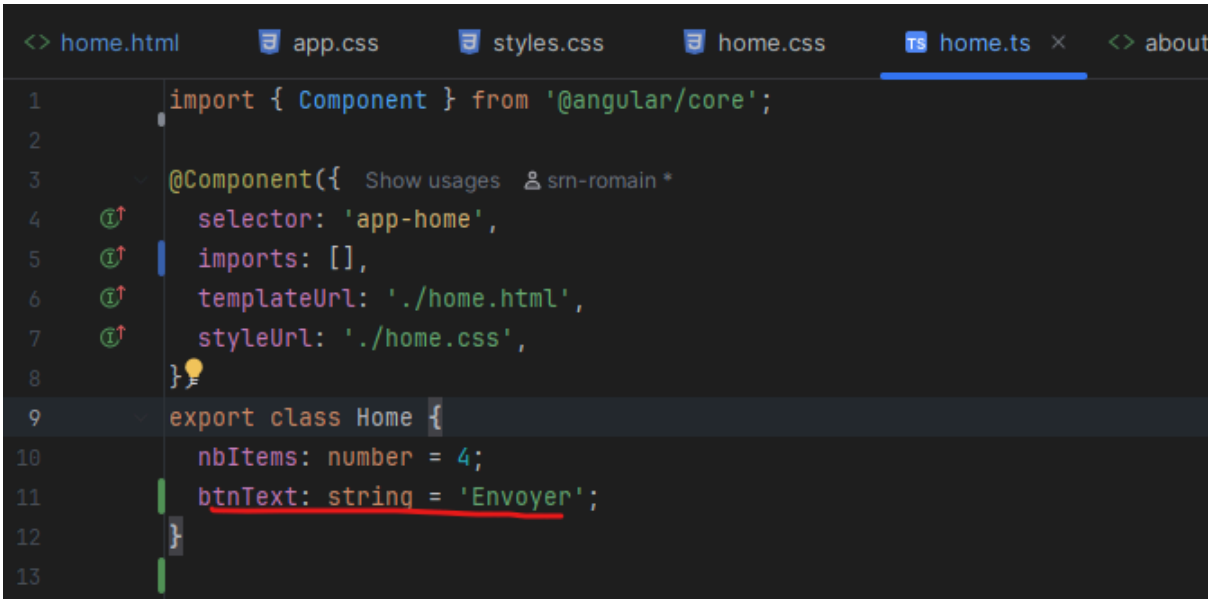


Ce qui donne au finale :

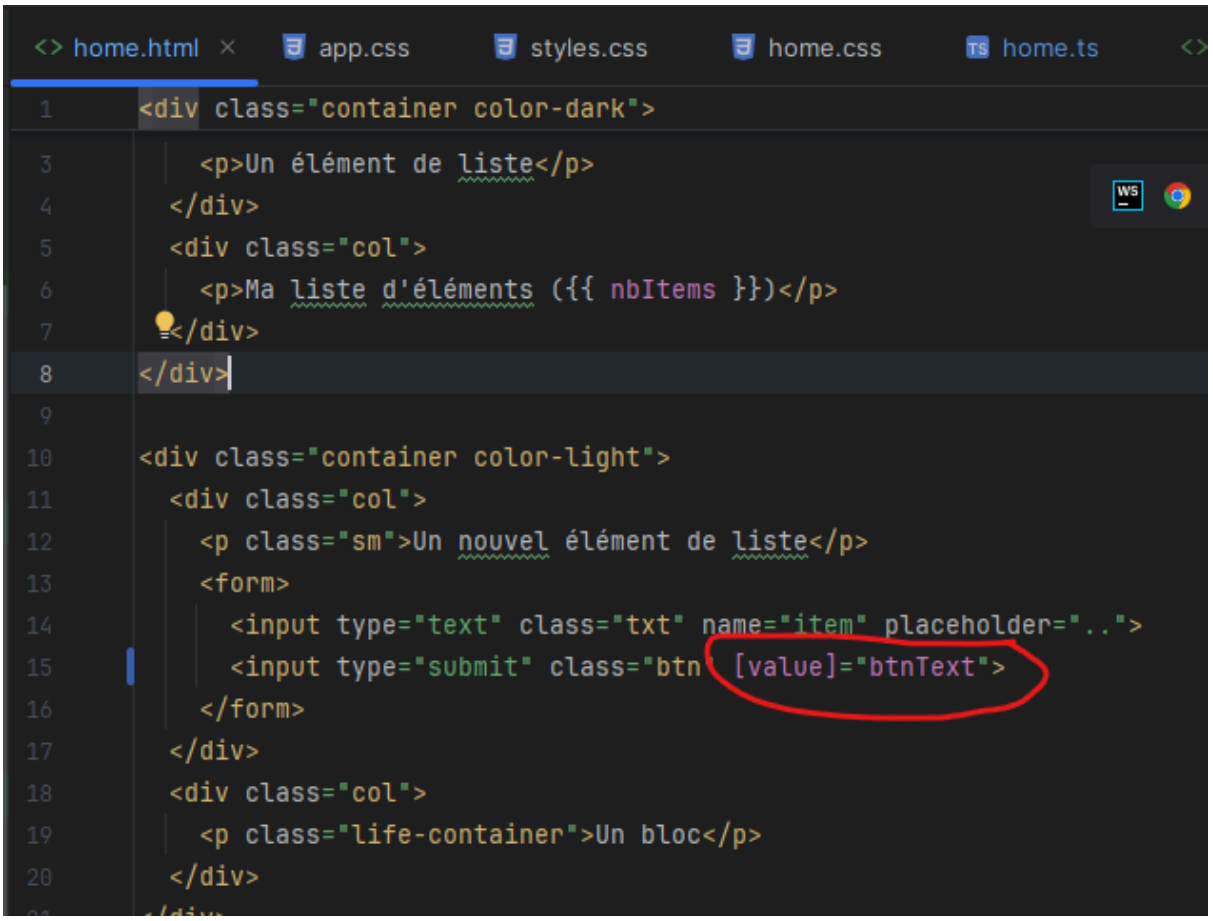


2.2 Comme pour la question précédente, on va rendre dynamique le texte du bouton du formulaire. Rajoutez une variable `btnText` de type `string` qui représentera le texte du bouton. Utilisez cette variable à l'aide d'un **property binding** pour afficher dynamiquement le texte du bouton.

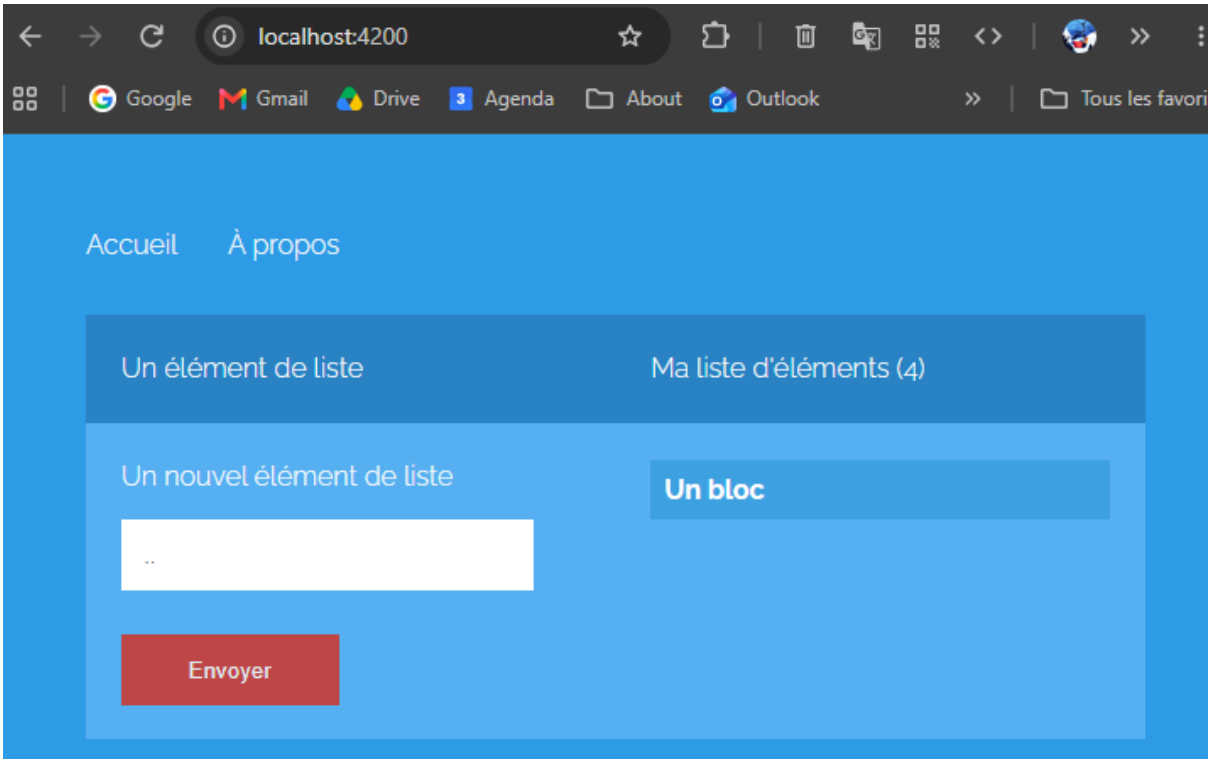
J'initialise la variable dans home.ts :



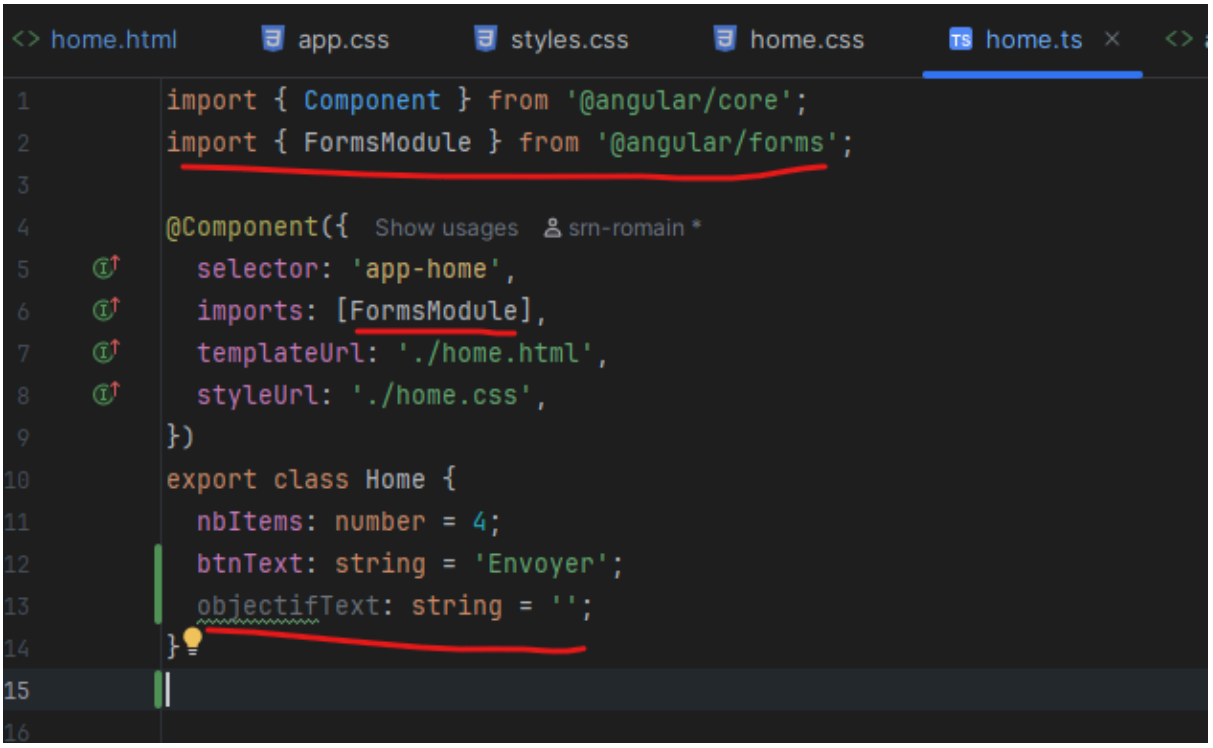
Et je l'utilise dans le home.html :



L'affichage lui n'a pas bougé :



2.3 Pour importer `FormsModule` dans un projet *Standalone*, il faut désormais l'impoter directemeent dans le composant concerné (ici dans le `HomeComponent`). Importez la classe `FormsModule` définie dans `'@angular/forms'`. Vérifiez que l'import dans la section *imports* est bien effectuée automatiquement ou faites le manuellement. Dans le fichier `home.ts`, rajoutez une variable `objectifText` de type string qui sera utilisée dans la question suivante pour faire la double liaison.



2.4 Dans votre fichier `home.component.html`, utilisez la double liaison avec `[(ngModel)]` pour mettre à jour automatiquement votre variable précédente `objectifText` et affichez le contenu de cette variable dans un paragraphe de façon à obtenir le même résultat que pour l'image suivante.



J'initialise ces variables :

```ts
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';

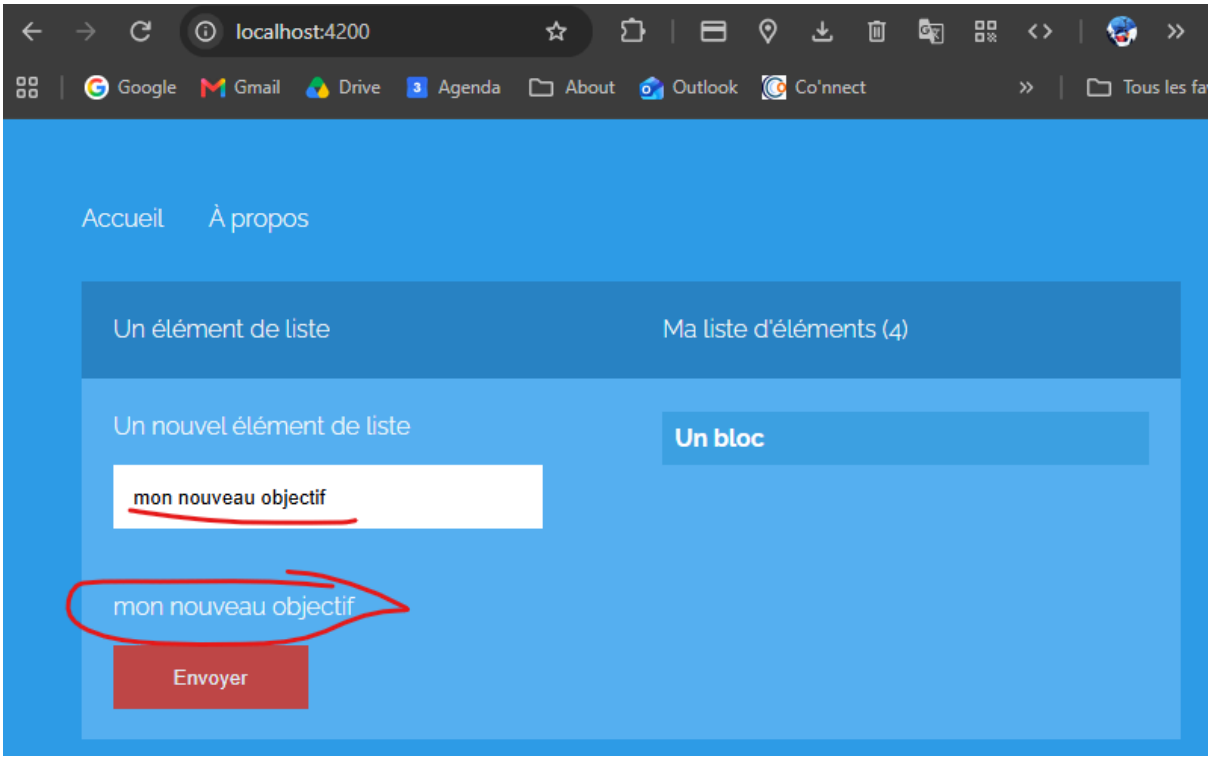
@Component({
  selector: 'app-home',
  imports: [FormsModule],
  templateUrl: './home.html',
  styleUrl: './home.css',
})
export class Home {
  nbItems: number = 4;
  btnText: string = 'Envoyer';
  objectifText: string = 'mon nouveau objectif';
}
```

Dans le home.html j'utilise ces variables :

```html
<div class="container color-dark">
  </div>
  <div class="col">
    <p>Ma liste d'éléments ({{ nbItems }})</p>
  </div>
</div>

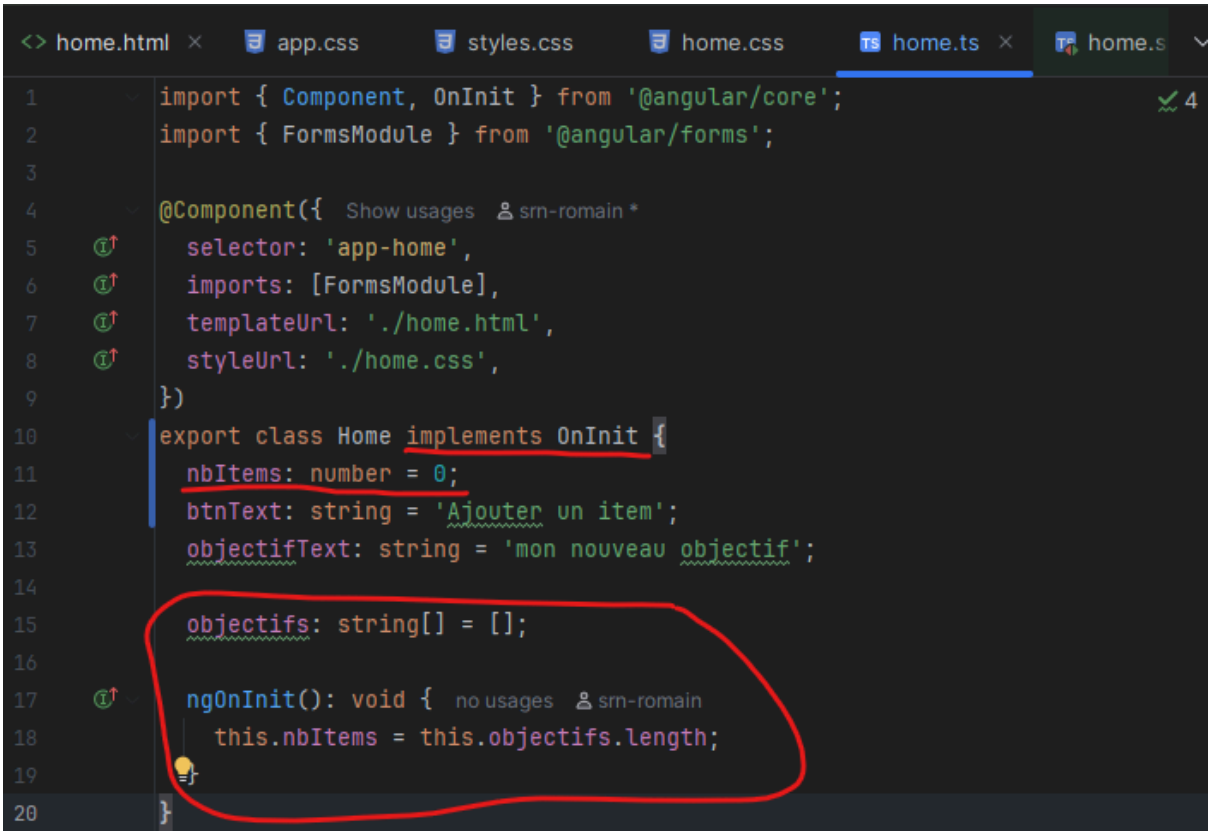
<div class="container color-light">
  <div class="col">
    <p class="sm">Un nouvel élément de liste</p>
    <form>
      <input type="text" class="txt" name="item" [(ngModel)]="objectifText">
      <p>{{ objectifText }}</p>
      <input type="submit" class="btn" [value]="btnText">
    </form>
  </div>
  <div class="col">
    <p class="life-container">Un bloc</p>
  </div>
</div>
```

En résultat, lors du chargement initiale de la page, j'ai bien comme l'image de la consigne :

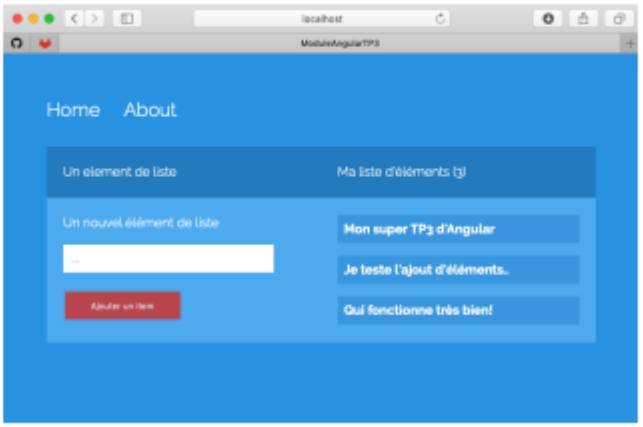


2.5 Dans le fichier `home.component.ts`, rajoutez un tableau `objectifs` initialisé à un tableau vide, et dans la méthode `ngOnInit` initialisez correctement la variable précédente `nbItems`.

J'ai donc procédé ainsi dans mon home.ts :



2.6 Toujours dans le même fichier, créez une nouvelle fonction `ajoutItem` qui rajoutera un nouvel élément au tableau (avec la méthode `push`) et qui mettra à jour la variable `nbItems`. Le nouvel élément rajouté sera défini à partir du texte écrit par l'utilisateur dans le formulaire.



J'initialise la fonction dans le home.ts :

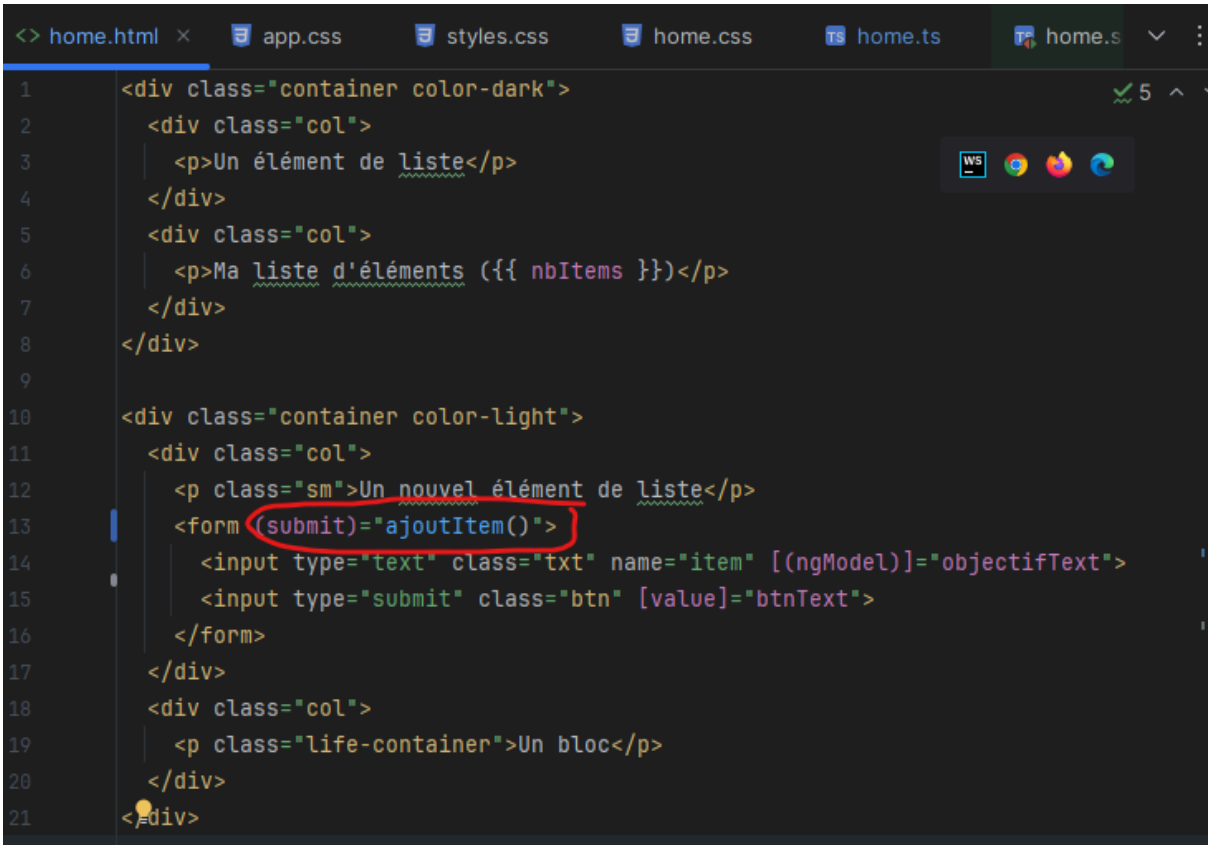
```ts
import { Component, OnInit } from '@angular/core';
import { FormsModule } from '@angular/forms';

@Component({
  selector: 'app-home',
  imports: [FormsModule],
  templateUrl: './home.html',
  styleUrl: './home.css',
})
export class Home implements OnInit {
  nbItems: number = 0;
  btnText: string = 'Ajouter un item';
  objectifText: string = 'mon nouveau objectif';
  objectifs: string[] = [];

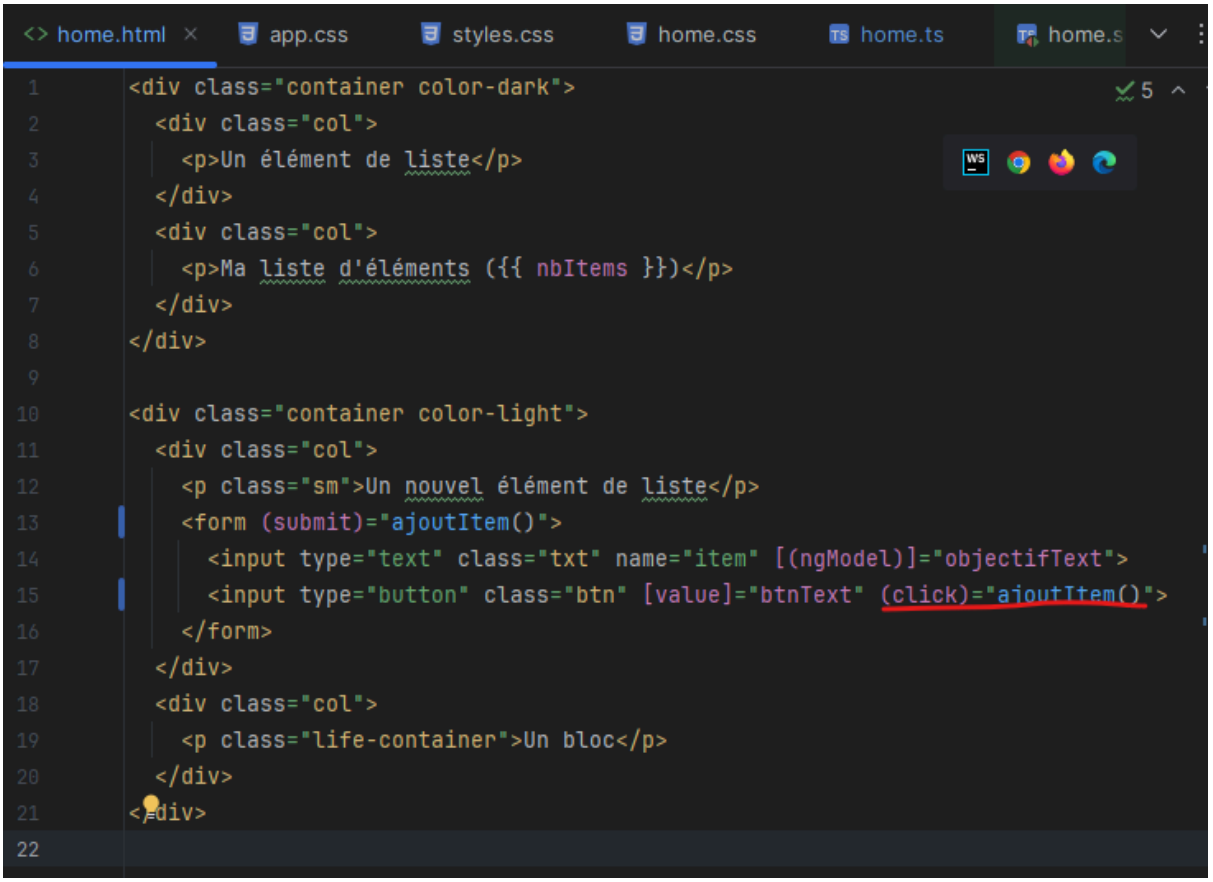
  ngOnInit(): void {
    this.nbItems = this.objectifs.length;
  }

  ajoutItem(): void {
    if (this.objectifText.trim() !== '') {
      this.objectifs.push(this.objectifText);
      this.nbItems = this.objectifs.length;
      this.objectifText = '';
    }
  }
}
```

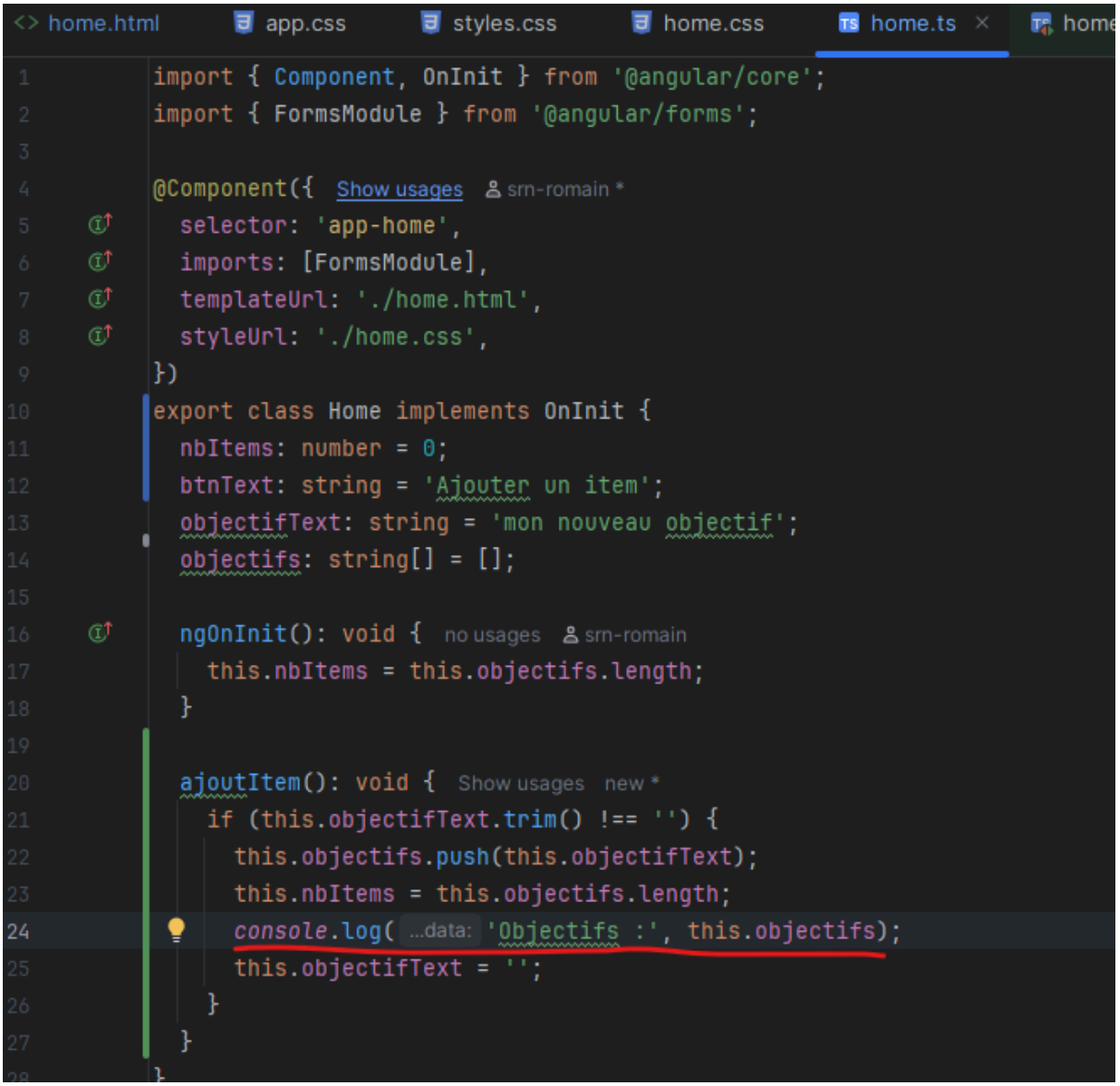
Et je l'utilise dans home.html :



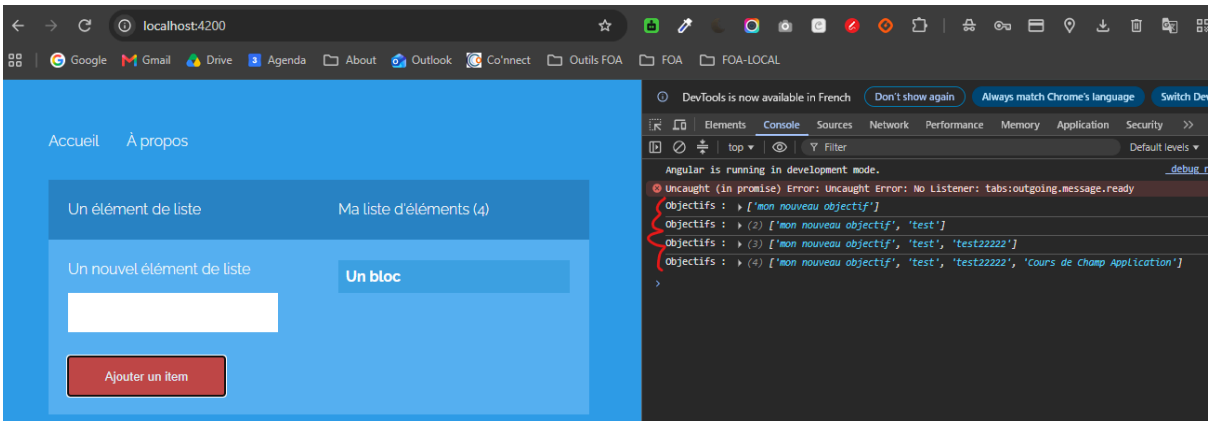
2.7 Associez le bouton du formulaire à un événement de *click* de souris qui déclenchera l'appel à la fonction précédente (`ajoutItem`). Vérifiez que l'appel de la fonction est bien fonctionnel.



Pour vérifier que cela focntionne bien j’ai ajouté un console.log :



J’ai bien les élément les uns après les autres dans la console :



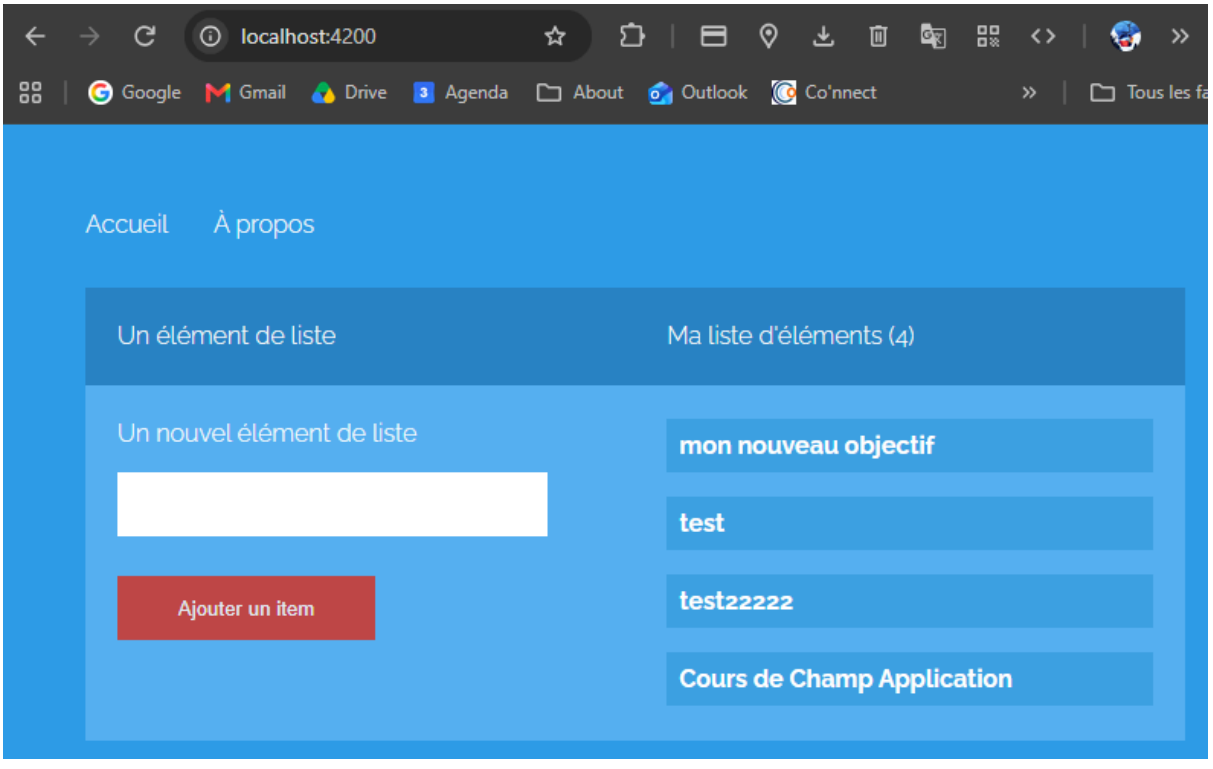
2.8 En utilisant un `@for`, modifiez la colonne du composant de classe `life-container` du fichier `home.component.html` de façon à afficher tous les éléments du tableau `objectifs`. Chaque élément sera de type : `<p class ="life-container"> element </p>`.

J'ajoute donc le for :

```
<div class="container color-dark">
  <div class="col">
    <p>Un élément de liste</p>
  </div>
  <div class="col">
    <p>Ma liste d'éléments ({{ nbItems }})</p>
  </div>
</div>

<div class="container color-light">
  <div class="col">
    <p class="sm">Un nouvel élément de liste</p>
    <form (submit)="ajoutItem()">
      <input type="text" class="txt" name="item" [(ngModel)]="objectifText">
      <input type="button" class="btn" [value]="btnText" (click)="ajoutItem()">
    </form>
  </div>
  <div class="col">
    @for (element of objectifs; track $index) {
      <p class="life-container">{{ element }}</p>
    }
  </div>
</div>
```

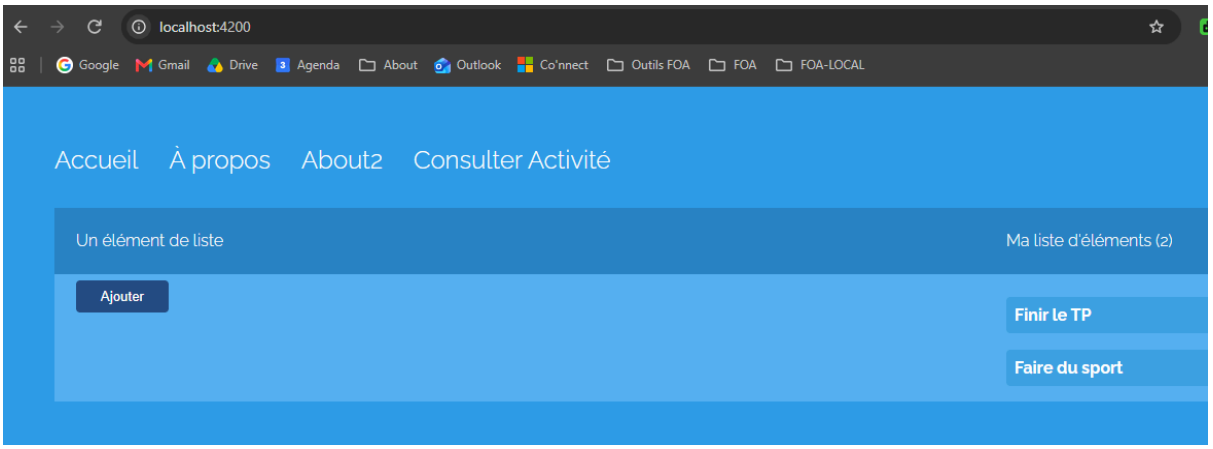
et cela donne bien :



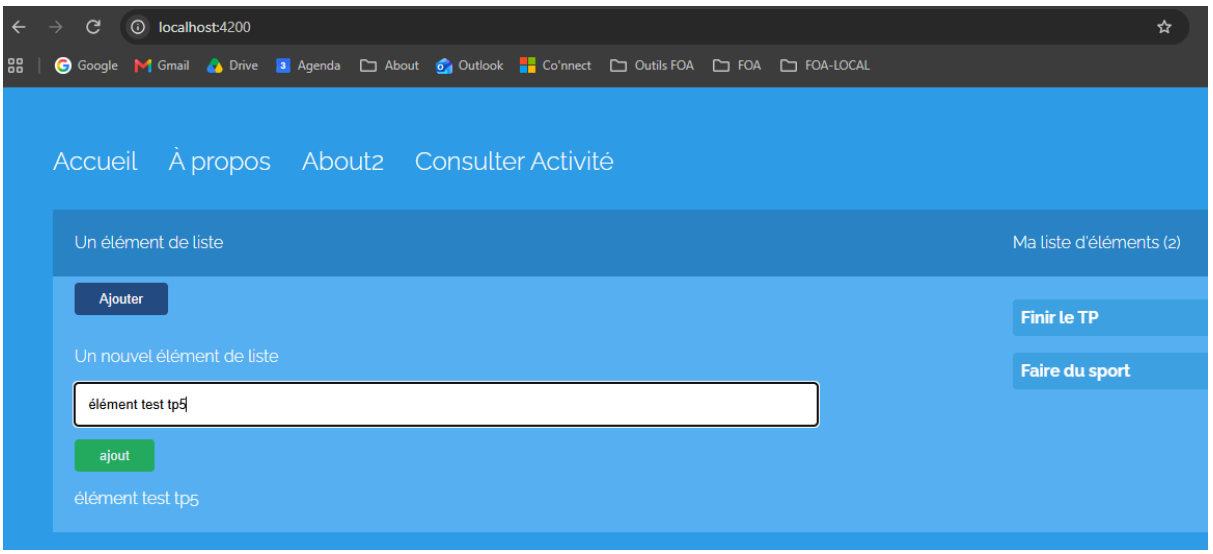
TP5 Compte rendu Champ d'Application

Romain SIEURIN - 18/09/2026

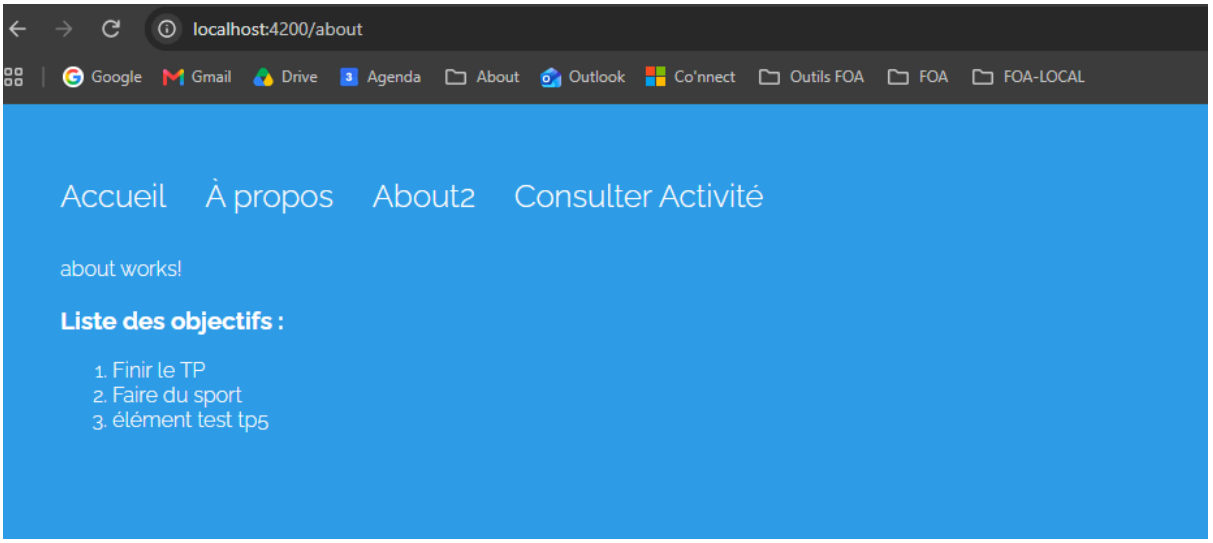
On repart pour débuté ce tp au début du tp 4. On obtiens donc cette affichage :



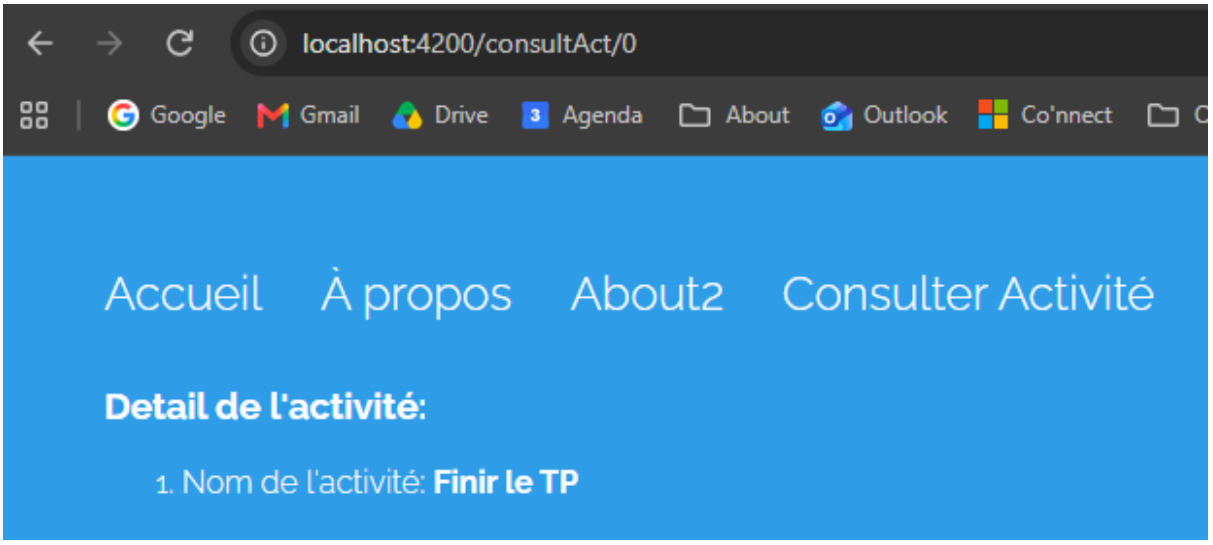
A partir du 1.4 de ce TP5, lorsque je clique sur ajouter, j'obtiens cette affichage :



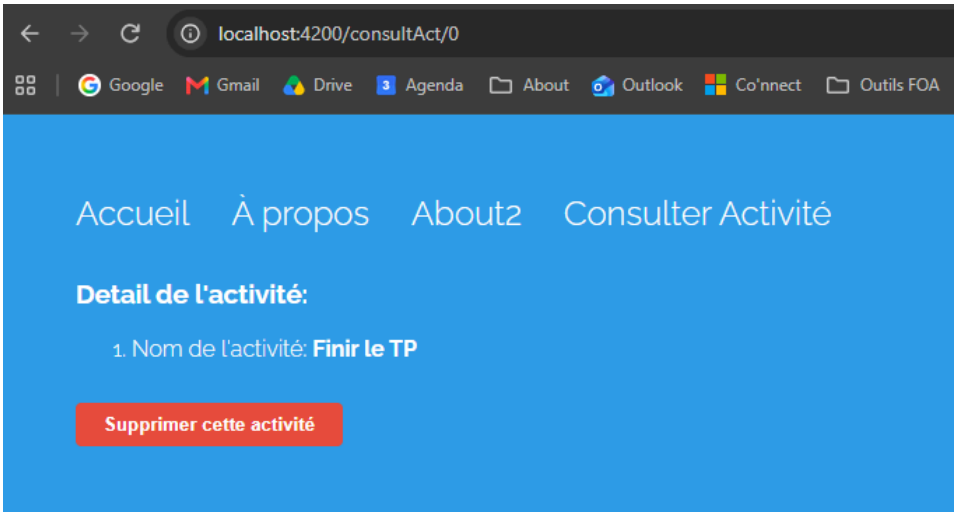
Puis après 1 seconde, je suis redirigé sur about comme on peut le voir sur la capture d'écran ci dessous. Avec la consigne 1.5, j'ai mis l'affichage des élément dans une liste à puce :



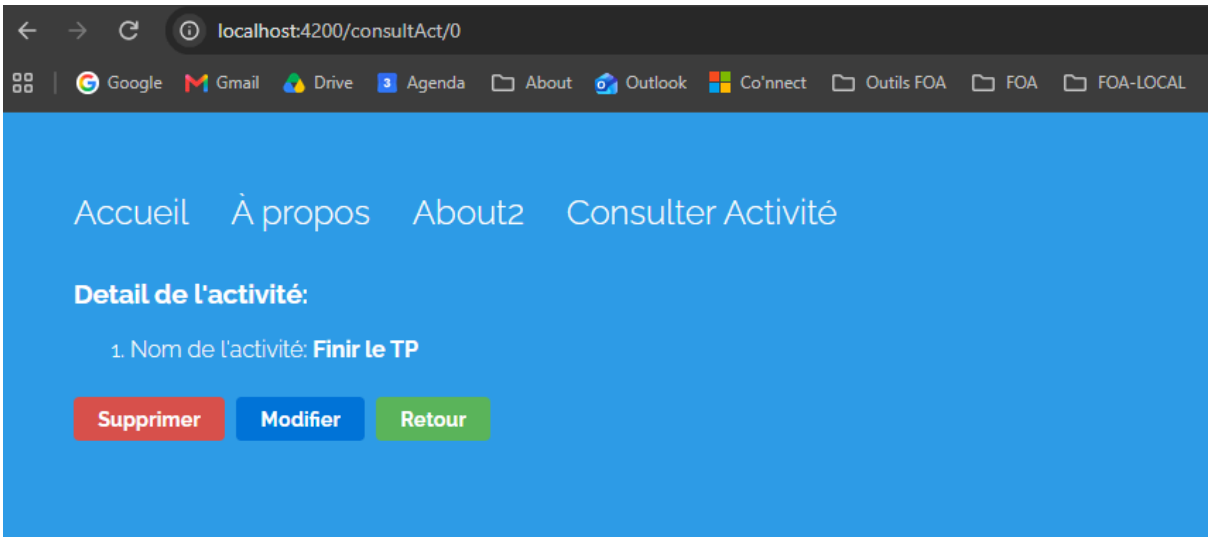
Ensuite, j'ai ajouté l'événement souris permettant d'ouvrir la page consultAct. ce qui me donne lorsque je clique sur un évènement de home :



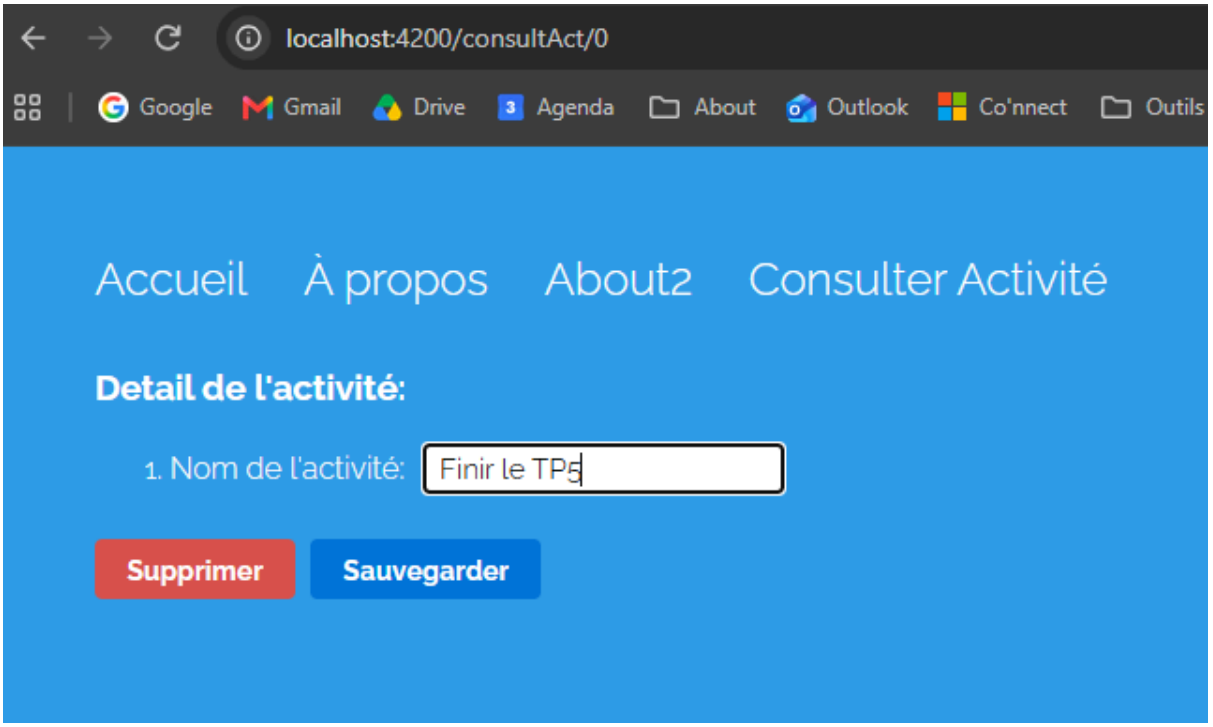
J'ai après fais la consigne 1.8 afin de pouvoir supprimé une activité :



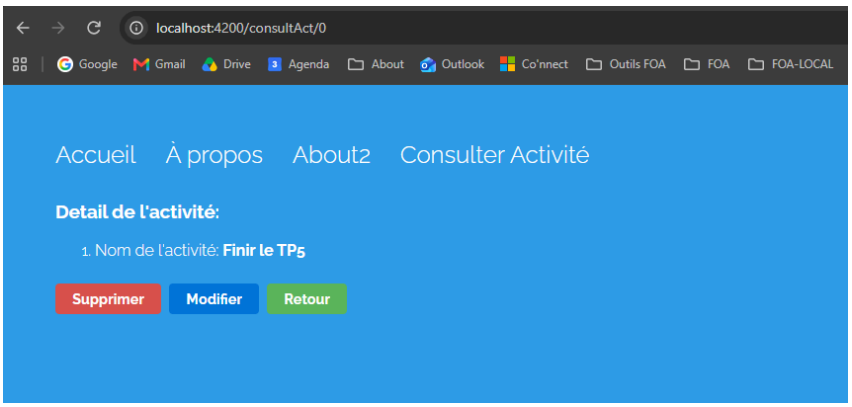
Puis pour la 1.9, j'ai ajouté un bouton afin de pouvoir modifier l'activité :



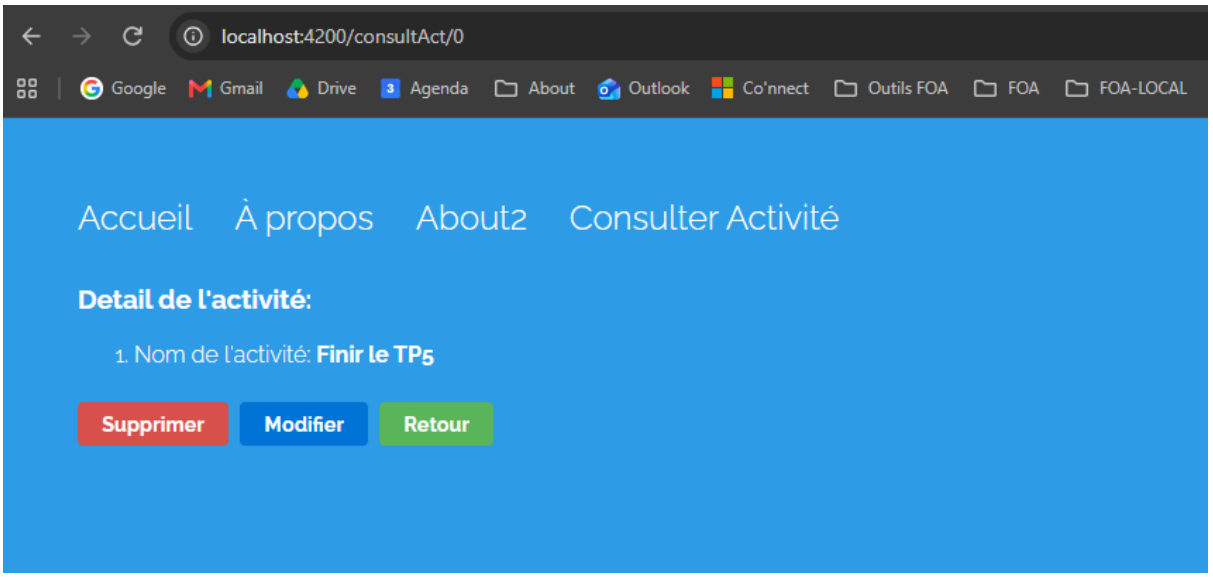
Et lorsque je clique sur modifier :



Puis sauvegarder :



J’ai bien au finale ma liste d’évènement modifier :



La page d’accueil suit cette modification :

